SYSTEM DESIGN / RELEASE ENGINEERING

Repaido deployment pipelines & flowmaps

A code-verified map of manual release selection, Cloud Build execution, production traffic changes, and the independent engineering console.

REVIEWED SOURCE

2950b4c3ed25cc41c36606886a26ac69d4a2a541

ENVIRONMENT

Google Cloud project: repaido
Region: us-central1

WEB

Firebase Hosting

Customer, worker and admin SPA routes

API

Cloud Run × 3

Main API, public work API, private worker

RULES

Firestore Rules

Emulator-tested security release

CONSOLE

Separate Cloud Run

Engineer-only architecture console

PUBLICLY OBSERVED LIVE STATE · 14 Oct 2026, 01:02 UTC

HOSTING

b421e09592a7

Build 4d2d4a26…

MAIN + WORK API

Same build as Hosting

Health endpoints: OK

ENGINEERING CONSOLE

f7446fcaae38

Four collectors: connected

The selective pipelines are present in this checkout. Compared with the live b421e095… site, this pending checkout spans multiple components, so its next customer release selects FULL. This PDF is not a complete cloud inventory export.

How a release target is chosen

The planner compares a reviewed commit with an explicitly supplied deployed baseline; it does not infer production from HEAD~1.

1. **Explicit request** — Manual batch; audit live triggers
2. **Pinned commit** — HEAD equals 40-character SHA
3. **Config + plan** — Generated YAML checked; diff classified
4. **Clean checkout** — No uncommitted source
5. **Source inventory** — Actual gcloud upload list
6. **No active build** — global + us-central1

PATH CLASSIFIER

web/ · firebase.json · .firebaserc	WEB	cloudbuild-web.yaml
backend/	API	cloudbuild-api.yaml
firestore.rules · rules tests	RULES	cloudbuild-rules.yaml
Multiple targets · unknown build input	FULL	cloudbuild.yaml
engineering/ only	CONSOLE	engineering/release.py
docs/ · android/ only	NONE	No customer pipeline

Fail closed: firestore.indexes.json is refused because no index publication step exists. Console and Android changes require separate release workflows; customer releases do not publish either.

The full production pipeline

Fifteen Cloud Build steps, in the exact order recorded in cloudbuild.yaml. The full batch is used for overlapping or unfamiliar changes.

01 / require-explicit-release

Manual release gate

Requires _RELEASE_APPROVED=true and pinned commit.

02 / check-build-history-access

Build history access

Checks release-order permissions.

03 / check-firestore-rules-access

Rules access

Checks existing Rules release + IAM.

04 / backend-tests

Backend tests

Python 3.12, Node, SQLite test mode.

05 / web-tests-and-build

Web tests + build

Rules emulator, npm tests, Vite, build-info.

06 / build-api

Build API image

Docker build from backend/.

07 / push-api

Push API image

Artifact Registry tag = Cloud Build ID.

08 / deploy-candidate

Main API candidate

No-traffic tag; health, Routes, Firestore/media.

09 / await-release-turn

Release ordering

Waits for older builds from same trigger.

10 / deploy-firestore-rules

Publish Rules

Compile, activate and verify changed Rules.

11 / deploy-work-services

Work candidates

Deploy work API + worker; configure/pause Scheduler.

12 / promote-backend

Promote main API

Traffic = verified main revision, 100%.

13 / promote-work-services

Promote work services

Traffic = verified work revisions; resume Scheduler.

14 / deploy-firebase-hosting

Deploy Hosting

Firebase Hosting only; SPA build + rewrites.

15 / verify-hosting-release

Verify public paths

Hosting + main/work API share this build ID.

Production change order: Rules → work candidate setup/Scheduler pause → main traffic → work traffic/Scheduler resume → Hosting. These are separate operations, not one atomic transaction.

Selective pipelines: deploy only what changed

Each lane retains the release gate and ordering check, while skipping unrelated production surfaces.

WEB

6 Cloud Build steps
No Cloud Run revision

- **Gate + history** — Manual override and build access.
- **Test + build** — pnpm; npm test; Vite. No Rules emulator.
- **Release turn** — Same-trigger ordering check.
- **Publish Hosting** — firebase deploy --only hosting.
- **Verify** — build-info commit + build ID.

API

11 Cloud Build steps
Three Run services

- **Gate + history** — Manual override and access.
- **Backend tests** — Pytest in SQLite test mode.
- **Image** — Docker build + explicit push.
- **Main candidate** — No-traffic health + config checks.
- **Work + traffic** — Work candidates, then main/work promotion.
- **Verify** — Main and work API build IDs.

RULES

6 Cloud Build steps
No image or Hosting

- **Gate + access** — Build history and Rules IAM.
- **Emulator** — Demo project; local Rules tests.
- **Release turn** — Same-trigger ordering check.
- **Publish** — Compile, activate and verify Rules.

Build IDs after a partial release: Hosting and API IDs can differ. The web verifier checks Hosting; the API verifier checks main and work API; the Rules publisher verifies the active ruleset.

Engineering console: independent release

The console has its own source archive, three-step Cloud Build, Cloud Run runtime identity, sign-in checks and promotion.

01

Resolve engineer accounts

Clean commit and no active builds; active Firebase accounts. Unavailable accounts may be deferred.

02

Preflight cloud access

Check Firebase config and Artifact Registry before a paid build.

03

Stage reviewed source

Filtered git archive; per-file SHA-256 manifest; no credentials.

04

IAM + engineer claims

Dedicated read-only runtime role; grant engineer claim.

05

Build console image

Manifest check, console tests, Docker build and push.

06

Deploy candidate

Dedicated Run service; prior service uses no-traffic tag.

07

Verify candidate

Health, live collectors, private API rejection, sign-in UI.

08

Authorize origins

Add returned Run origins to Firebase Auth allowlist.

09

Promote + reverify

100% verified revision; check each public origin.

OBSERVED CONSOLE

Public /health at 14 Oct 2026, 01:02 UTC reports revision **f7446fcaae38**, build **5765c997…**, four collectors connected, and engineer account checks passing. The current selective-release checkout is separate and has not been published to the customer site.

Production topology: build plane and serving plane

Solid arrows are serving or delivery paths; dashed arrows represent build/deployment supply.

BUILD / DEPLOYMENT PLANE

Reviewed Git commit
Clean checkout + manual release

Cloud Shell operator
Planner, inventory, active-build check

Cloud Build
Target YAML + tests + build ID

Artifact Registry
API / console images

Cloud Run
Image revision, candidate, traffic

SERVING / EVENT PLANE

Customer web client
Reads SPA; calls /api/* through Hosting.

Firebase Hosting
repaido.web.app; SPA routes + API rewrites.

Main API
repaido-api · /api/**

Routes + Secret Manager
Main candidate requires Routes key readiness.

Work API
repaido-work-api · work/companies/placements

Firestore
Operational records for main and work services.

Cloud Scheduler
repaido-work-updates · every minute.

Private work worker
OIDC caller; POST /internal/work/dispatch.

Notifications
Worker sends opted-in work updates via FCM.

Engineering console
Separate Run service; engineer sign-in; read-only cloud observations.

Cloud observations
Asset, Monitoring, Logging, Trace.

Hosting routing: /api/repaidians/work/**, /companies/** and /placements/** → work API; /api/** → main API; other paths → index.html. Hosting publishes directly from Cloud Build, without an image. Main API also uses private community media storage.

Service contracts and trust boundaries

These limits and identities come from the deployment commands; the main API retains other existing runtime settings.

MAIN API · repaido-api

Existing service required. Same backend image. New candidate uses --no-traffic and a build tag. Sets REPAIDO_BUILD_ID, Firestore storage mode and the Google Routes secret. Checks health, routing readiness and durable community media before traffic promotion.

WORK API · repaido-work-api

work_service:app from the same image. Public service; inherits the main runtime identity. 1 CPU · 512 MiB · concurrency 64 · timeout 60 s · min 0 / max 20. Mail ownership and Cloudflare Stream secrets are removed.

PRIVATE WORKER · repaido-work-worker

work_worker:app from the same image. IAM-protected; the Scheduler caller is granted invoker. 1 CPU · 512 MiB · concurrency 1 · timeout 240 s · min 0 / max 2. Push enabled; signing-key secrets filtered.

ENGINEERING CONSOLE

repaido-engineering uses a dedicated read-only identity. Public ingress, but engineering APIs require an engineer account. 1 CPU · 512 MiB · concurrency 40 · timeout 180 s · min 0 / max 1; CPU throttled.

FIREBASE HOSTING

Deploys web/dist with build-info.json. SPA routes serve index.html. Specific work/companies/placements API paths rewrite to the work API; other /api/** paths to the main API. No Docker image is needed for a web-only release.

FIRESTORE RULES

The Rules API publisher checks the existing release, compares source, compiles changed Rules, activates the ruleset and reads back its identity. Firestore indexes have no automated step and cause the selective planner to stop.

PRIVATE DISPATCH PATH

Cloud Scheduler → OIDC scheduler identity → Private worker POST → FCM delivery

Job **repaido-work-updates** runs every minute in Asia/Kolkata with a 240-second deadline, up to three retries and 30–300-second backoff. The work deployment pauses it during candidate setup; promotion resumes it. New work services may serve on first creation because --no-traffic is conditional on a prior service.

Failure boundaries and operational controls

The pipeline verifies candidates before promotion, but production updates are sequential and are not automatically rolled back.

WINDOW 1

Before candidate

Tests or image build fail
No new production traffic. Existing site stays live.

WINDOW 2

Candidate checks

Main/work candidate fails
Main traffic stays; first-time work service creation can serve immediately.

WINDOW 3

Rules published

Later step fails
New Firestore Rules can already be active.

WINDOW 4

Work setup

Failure before work promotion
Scheduler may remain paused; inspect and restore delivery.

WINDOW 5

Traffic / Hosting

Failure during promotion
Main, work and Hosting may serve different builds.

RESPONSE MAP

Cloud Run	Inspect tagged/serving revisions and restore a previously verified revision if needed.
Hosting	Restore a prior Hosting release if web publication or rewrites are wrong.
Rules	Restore a previously reviewed Rules release if access changes are wrong.
Scheduler	Check the private work-updates job is active after a work deployment failure.

COST CONTROL THAT THE REPO ACTUALLY IMPLEMENTS

The repo provides a manual release gate (false by default) and component-scoped builds. Disabling the live production push trigger is recommended but was not verified here. The root source archive is still uploaded for selective releases; runtime, storage and network costs continue. Measure savings in Billing Reports.

Operator path, source trace and evidence limits

Use the plan before any approved release. This document describes checked-in behavior and the limited public endpoints read for verification.

1 Review the exact change set

Identify the real deployed baseline. The plan compares two full Git SHAs, classifies changed paths and reports the selected config. A docs-only change does not publish the customer app.

2 Submit only an authorized batch

Use the same reviewed SHA and baseline with --release in authenticated Cloud Shell. The helper checks clean source, upload inventory and active builds before submitting Cloud Build.

OFFLINE PLAN

```
bash scripts/release-batch.sh --plan <reviewed-40-char-sha> --changed-from <deployed-40-char-sha>
```

MANUAL RELEASE · only after a release request

```
bash scripts/release-batch.sh --release <reviewed-40-char-sha> --changed-from <deployed-40-char-sha>
```

CHECKED SOURCE

- scripts/release-batch.sh + scripts/plan-release.py
- cloudbuild.yaml + cloudbuild-{web,api,rules}.yaml
- scripts/deploy-work-services.py + scripts/deploy-firestore-rules.py
- firebase.json + backend/Dockerfile
- engineering/release.py + engineering/cloudbuild.yaml

Evidence limit: public health checks confirmed the build IDs shown on page 1, but this document does not claim a complete live IAM, network, billing, secret, or capacity audit. Source files define the intended pipeline; a Cloud Shell audit is required to compare every live resource with it.